

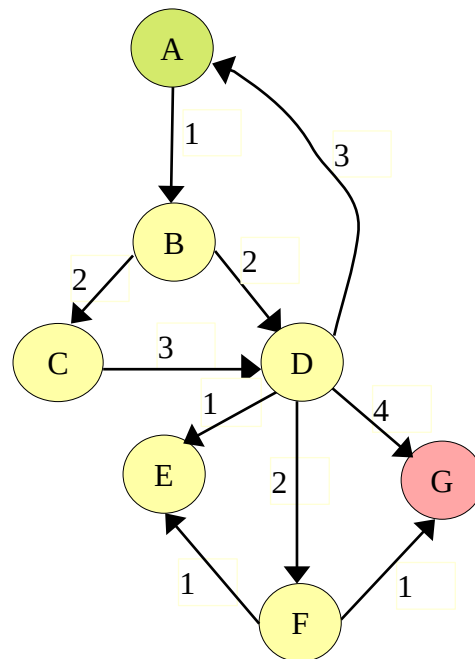
TP N 03 : Les Algorithmes de Recherche Informés/non-Informés

I. But du TP

Le présent TP constitue la suite du TP précédent, dont l'objectif était de mettre en pratique les algorithmes de recherche étudiés en cours. Il est consacré à l'implémentation des algorithmes de recherche non informés IDS et UCS, ainsi que de l'algorithme informé A*.

Partie I : Les algorithmes DLS, IDS

Soit l'espace de recherche du TP précédent :



Cet espace de recherche peut être représenté par un dictionnaire. Dans ce dictionnaire, les clés correspondent aux états (nœuds) de l'espace, tandis que les valeurs associées à chaque clé représentent une liste des successeurs (fils) de cet état.

```

MyResearchSpace={
  'A': ['B'],
  'B': ['C','D'],
  'C': ['D'],
  'D': ['A','E','F','G'],
  'E': [],
  'F': ['E','G'],
  'G': [],
}
  
```

Pour implémenter l'algorithme IDS, nous allons commencer par l'implémentation de l'algorithme DLS (voir le cours). Pour ce faire, utilisez le code et les fonctions suivantes :

- La classe « Node » qui représente un état dans l'espace de recherche :

TP N 03 : Les Algorithmes de Recherche Informés/non-Informés

```
# la classe nœud ou etat
class Node:
    def __init__(self, name, parent, action):
        self.name = name #Le nom de l'etat
        self.parent = parent # Le père d'un nœud
        self.action = action # L'action qui génère le nœud (optionel)
```

- La fonction «**gener_successeur(etat)**» qui prend en paramètre un état, et retourne une liste de ses successeurs (selon l'espace de recherche précédent).

```
def gener_successeur(etat):
    if etat.name in myResearchSpace:
        listSucc=myResearchSpace[etat.name]
        if len(listSucc) > 0:
            return [Node(x,etat,"") for x in listSucc]
        else:
            return []
```

- La fonction «**test_etat_final(C,F)**», qui prend en paramètre, un état 'C' et une liste d'états F représentant les états finaux. La fonction retourne 'True' si C appartient à F, et 'False' sinon.

```
def test_etat_final(C,F):
    if C.name in F:
        return True
    else: return False
```

- La fonction **getSolution(N)**, qui prend en paramètre un état N (un objet de type *Node*) et retourne une liste des noms des états depuis l'état initial I jusqu'à l'état N.

```
def getSolution(N):
    solution=[]
    parent=N.parent
    if parent is not None: solution=[parent.name,N.name]
    else: return [N.name]
    while parent.parent:
        solution.insert(0,parent.parent.name)
        parent=parent.parent
    return solution
```

1. Écrire la fonction «**dls_search1(I,F,L, OV)** » qui prend en paramètres, un état initial 'I', une liste d'états finaux 'F', la limite de la profondeur 'L' et une liste 'OV' pour stocker l'ordre des nœuds visités. La fonction retourne trois éléments : **un booléen** indiquant si l'algorithme a trouvé l'un des états finaux, la **liste des états visités** « **OV** », et enfin une liste représentant **la solution** (le chemin depuis l'état initial jusqu'à l'état final).
2. Tester votre fonction en utilisant les paramètres suivants en analysant l'ordre de visite à chaque test:

```
I=Node('A',None,"")
dls_search(I,['A'],0,[])
```

TP N 03 : Les Algorithmes de Recherche Informés/non-Informés

```
dls_search(l,['G'],0,[])
```

```
dls_search(l,['G'],1,[])
```

```
dls_search(l,['G'],2,[])
```

```
dls_search(l,['G'],3,[])
```

- Donnez l'ordre de visite, et la solution trouvée par l'algorithme IDS depuis l'état 'A' jusqu'à l'état 'G'.
- En utilisant la fonction la fonction « `dls_search` », implémenter la fonction « `IDS_search(l,F)` » qui prend en paramètres, un nom d'état initial 'I' et une liste d'états finaux 'F' (exemple `IDS_search('A',['G'])`). La fonction retourne trois éléments : **un booléen** indiquant si l'algorithme a trouvé l'un des états finaux, la **liste des états visités de chaque itération (de chaque limite)**, et enfin une liste représentant la **solution** (le chemin depuis l'état initial jusqu'à l'état final).
- Tester votre fonction en utilisant les paramètres suivants :

```
IDS_search('A',['A'])
```

```
IIDS_search('A',['C'])
```

```
IIDS_search('A',['D'])
```

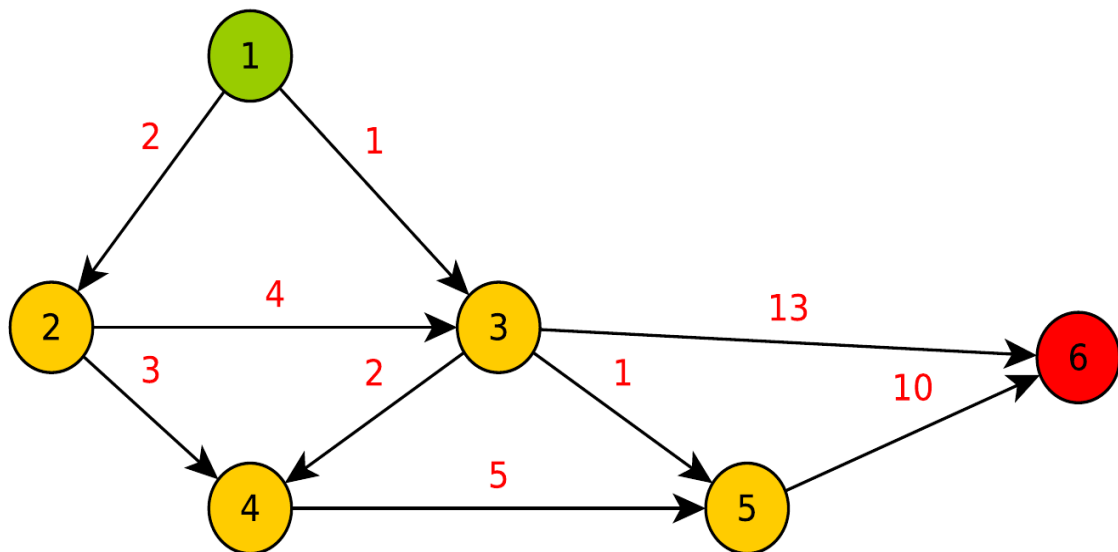
```
IDS_search('A',['E'])
```

```
IDS_search('A',['G'])
```

Partie II : Les algorithmes UCS (Uniform Cost Search) et A*

L' algorithme UCS (Uniform Cost Search)

Soit l'espace de recherche suivant :



TP N 03 : Les Algorithmes de Recherche Informés/non-Informés

Le coût de déplacement d'un état à un autre est indiqué sur les arcs. Cet espace peut être représenté de la manière suivante :

```
myResearchSpace = {
  1: [(2, 2), (3, 1)],
  2: [(3, 4), (4, 3)],
  3: [(4, 2), (5, 1), (6, 13)],
  4: [(5, 5)],
  5: [(6, 10)],
  6: [],
}
```

- Nous modifions la classe « Node » en ajoutant un attribut 'coût', qui représente le coût pour atteindre un état N depuis l'état initial. La classe « Node » devient alors la suivante :

```
class Node:
    def __init__(self, name, parent, cout, action):
        self.name = name #Le nom de l'etat
        self.parent = parent # Le père d'un nœud
        self.cout=cout #le coût depuis l'état initial
        self.action = action # L'action qui génère le nœud (optionnel)
```

1. Modifiez la fonction « `gener_successeur(etat)` » pour qu'elle génère les successeurs d'un état en prenant en compte la modification de la classe « Node ». Autrement dit, elle doit générer des successeurs avec leurs coûts depuis l'état initial."

2. En adaptant l'algorithme « dfs » suivant :

```
def dfs_search(I,F):
    open=[]
    closed=[]
    ordreVisite=[]
    open.append(Node(I, None, ""))
    while open:
        N = open.pop()
        ordreVisite.append(N.name)
        if test_etat_final(N, F):
            return True,ordreVisite,getSolution(N)
        else:
            closed.append(N)
            succ = gener_successeur(N)
            if len(succ) > 0:
                for s in succ:
                    if s.name not in [x.name for x in open] and s.name not in [x.name for x in closed]:
                        open.append(s)
    return False,ordreVisite,[]
```

Écrire la fonction « `ucs_search(I,F)` » qui prend en paramètres un état initial 'I' et une liste d'états finaux 'F' (par exemple : `ucs_search(1, [6])`). Elle retourne **quatre éléments** : un **booléen** indiquant si l'algorithme a trouvé la solution, une **liste des états visités** durant la recherche, une liste représentant **la solution**, et enfin le **coût** de cette solution. Vous pouvez utiliser les indications suivantes :

TP N 03 : Les Algorithmes de Recherche Informés/non-Informés

a) L'algorithme UCS peut avoir la même implémentation que « DFS » ou « BFS ». La différence réside dans la structure « open », qui devient une liste triée en fonction du coût.

b) Pour trier une liste « list » d'objets de type « Node » selon le coût, vous pouvez utiliser le code suivant :

```
list.sort(key=lambda x: x.cout) pour un ordre croissant
```

```
list.sort(key=lambda x: x.cout, reverse=True) pour un ordre décroissant
```

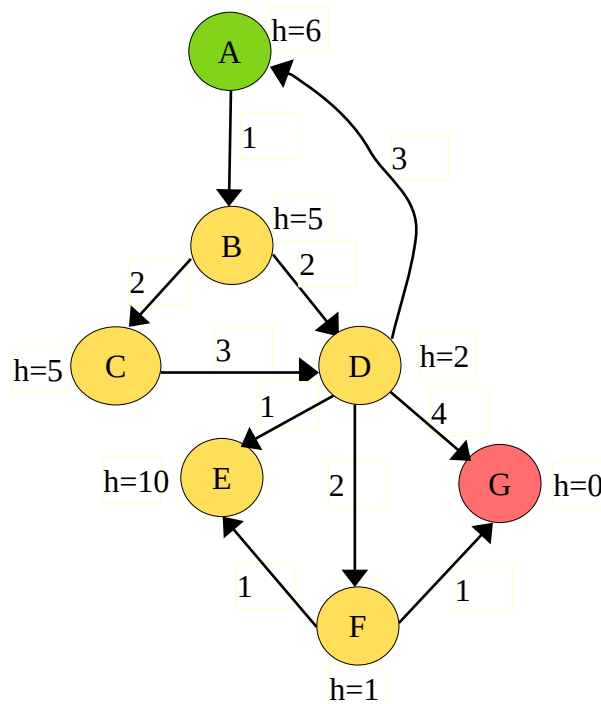
c) Dans UCS, un état peut être revisité (réintroduit dans « open ») si son coût est inférieur à celui de l'état portant le même nom dans « closed ».

d) Si un état « N1 » est généré avec un coût inférieur à celui d'un état « N » portant le même nom et déjà présent dans « open », alors « N » est remplacé par « N1 » dans « open »."

3. Testez votre implémentation avec l'exemple : `ucs_search(1,[6])`

L'algorithme A*

Soit l'espace de recherche suivant. Le nœud A est l'état initial et G est l'état final. La valeur heuristique $h(n)$ est indiquée à chaque nœud.



1. Testez votre implémentation UCS en utilisant cet espace de recherche et Donnez l'ordre de visite, la solution et le coût de la solution.

2. Pour implémenter l'algorithme nous modifions la classe « Node » en ajoutant un attribut « coutf » qui stocke aussi la valeur de la fonction « $f = g + h$ » de la manière suivante :

TP N 03 : Les Algorithmes de Recherche Informés/non-Informés

```
class Node1:
    def __init__(self, name, parent, cout, coutf, action):
        self.name = name #Le nom de l'etat
        self.parent = parent # Le père d'un nœud
        self.cout=cout #le coût depuis l'état initial
        self.coutf = coutf # le coût depuis l'état initial
        self.action = action # L'action qui génère le nœud (optionel)
```

- a) Modifiez la fonction « `gener_successeur(etat)` » pour qu'elle génère les successeurs d'un état en prenant en compte la modification de la classe « Node ». Autrement dit, en plus de générer des successeurs avec leurs coûts depuis l'état initial, elle doit générer aussi la valeur de la fonction « f » dans l'attribut « coutf ».
- b) En modifiant la fonction « `ucs_search(I,F)` », implémentez la fonction « `astar_search(I,F)` » en triant les éléments dans la liste open et en effectuant les comparaisons en fonction de l'attribut « coutf » au lieu de « cout ».
- c) Testez votre implémentation avec l'exemple : `astar_search('A',['G'])`. Est-ce que cela donne le même résultat en termes de solution, de coût et d'ordre de visite que UCS ? Expliquez.